

```

/**
 * =====
 * Slops Saloon Fantasy Football MVP (SSFFMVP)
 * Platform Integration Layer – v3 Production
 * =====
 *
 * Handles OAuth and live data for:
 *   - Sleeper (no OAuth – public API by username)
 *   - Yahoo (OAuth 2.0 with PKCE)
 *   - ESPN (cookie-based ESPN_S2 + SWID)
 *
 * v3 security hardening in this file:
 *   [2] Redis cache on standings (5 min TTL) and roster (2 min TTL)
 *       100 users in same league = 1 API call, not 100.
 *   [3] Supabase Vault encryption on all sensitive credentials:
 *       Yahoo access_token, Yahoo refresh_token, ESPN espn_s2
 *       Only Vault secret_id UUIDs are stored in DB columns.
 *       Decryption happens at query time via vaultDecrypt().
 *
 * Routes:
 *   POST /api/auth/sleeper/connect
 *   GET /api/auth/yahoo/authorize
 *   GET /api/auth/yahoo/callback
 *   POST /api/auth/espn/connect
 *   GET /api/league/standings
 *   GET /api/league/roster
 * =====
 */

"use strict";

const express = require("express");
const crypto = require("crypto");
const { createClient } = require("@supabase/supabase-js");
const { Redis } = require("@upstash/redis"); // npm i @upstash/redis

const router = express.Router();
const supabase = createClient(process.env.SUPABASE_URL, process.env.SUPABASE_SERV

// [2] Redis client – Upstash serverless (free tier covers SSFFMVP at launch scal
const redis = new Redis({
  url: process.env.REDIS_URL,
  token: process.env.REDIS_TOKEN,
});

```

```
// — ENV —————
const YAHOO_CLIENT_ID      = process.env.YAHOO_CLIENT_ID;
const YAHOO_CLIENT_SECRET = process.env.YAHOO_CLIENT_SECRET;
const YAHOO_REDIRECT_URI   = process.env.YAHOO_REDIRECT_URI;
const APP_BASE_URL         = process.env.APP_BASE_URL;

/* =====
[3] VAULT HELPERS
vaultDecrypt: given a secret_id UUID stored in the DB,
returns the plaintext secret via the vault_decrypt_secret RPC.
The RPC wraps vault.decrypted_secret() with security definer
so the service role can call it without exposing the raw vault.

vaultCreate: encrypts a new secret, returns its secret_id UUID
to be stored in the DB column instead of the plaintext value.

One-time Supabase SQL setup (run in SQL editor):
-----
create extension if not exists supabase_vault;

create or replace function vault_decrypt_secret(secret_id uuid)
returns jsonb language sql security definer as $$
  select jsonb_build_object(
    'decrypted_secret', vault.decrypted_secret(secret_id)
  );
$$;

create or replace function vault_create_secret(
  secret text, name text, description text default ''
) returns uuid language sql security definer as $$
  select vault.create_secret(secret, name, description);
$$;

create or replace function vault_update_secret(
  secret_id uuid, new_secret text
) returns void language sql security definer as $$
  update vault.secrets set secret = new_secret where id = secret_id;
$$;
----- */
async function vaultDecrypt(secretId) {
  if (!secretId) return null;
  const { data, error } = await supabase.rpc("vault_decrypt_secret", { secret_id:
  if (error) throw new Error(`Vault decrypt failed (${secretId}): ${error.message}
  return data?.decrypted_secret ?? null;
}

async function vaultCreate(secret, name, description = "") {
```

```

    const { data, error } = await supabase.rpc("vault_create_secret", { secret, name });
    if (error) throw new Error(`Vault create failed (${name}): ${error.message}`);
    return data; // returns UUID of the new secret
}

```

```

async function vaultUpdate(secretId, newSecret) {
    const { error } = await supabase.rpc("vault_update_secret", { secret_id: secretId, new_secret: newSecret });
    if (error) throw new Error(`Vault update failed (${secretId}): ${error.message}`);
}

```

```

/* =====
   █████ SLEEPER █████
   Sleeper uses a fully public API – no OAuth required.
   Users authenticate by username. We resolve username → user_id,
   then pull their leagues and roster.
   Docs: https://docs.sleeper.com/
   ===== */

```

```

const SLEEPER = "https://api.sleeper.app/v1";

```

```

class SleeperClient {
    async getUser(username) {
        const res = await fetch(`${SLEEPER}/user/${username}`);
        if (!res.ok) throw new Error(`Sleeper user not found: ${username}`);
        return res.json();
        // Returns: { user_id, username, display_name, avatar }
    }

    async getUserLeagues(userId, season = new Date().getFullYear()) {
        const res = await fetch(`${SLEEPER}/user/${userId}/leagues/nfl/${season}`);
        if (!res.ok) throw new Error("Could not fetch Sleeper leagues");
        return res.json();
        // Returns: [{ league_id, name, total_rosters, scoring_settings, ... }]
    }

    async getLeagueRosters(leagueId) {
        const res = await fetch(`${SLEEPER}/league/${leagueId}/rosters`);
        if (!res.ok) throw new Error("Could not fetch rosters");
        return res.json();
        // Returns: [{ roster_id, owner_id, players, starters, settings: { wins, loss
    }

    async getLeagueUsers(leagueId) {
        const res = await fetch(`${SLEEPER}/league/${leagueId}/users`);
        if (!res.ok) throw new Error("Could not fetch league users");
        return res.json();
        // Returns: [{ user_id, display_name, avatar, metadata: { team_name } }]
    }
}

```

```

async getLeagueMatchups(leagueId, week) {
  const res = await fetch(`${SLEEPER}/league/${leagueId}/matchups/${week}`);
  if (!res.ok) throw new Error("Could not fetch matchups");
  return res.json();
}

async getPlayerInfo() {
  // Sleeper provides a full player DB - cache this, it's large (~5MB)
  const res = await fetch(`${SLEEPER}/players/nfl`);
  if (!res.ok) throw new Error("Could not fetch player data");
  return res.json();
  // Returns: { "player_id": { full_name, position, team, status, injury_status
}

/**
 * Build standings from rosters + users.
 * Returns normalized format used by the SSFFMVP frontend.
 */
async buildStandings(leagueId, myUserId) {
  const [rosters, users] = await Promise.all([
    this.getLeagueRosters(leagueId),
    this.getLeagueUsers(leagueId),
  ]);

  const userMap = Object.fromEntries(users.map(u => [u.user_id, u]));

  const standings = rosters
    .map(r => {
      const user = userMap[r.owner_id] || {};
      const wins = r.settings?.wins || 0;
      const losses = r.settings?.losses || 0;
      const fpts = r.settings?.fpts || 0;
      const fptsD = r.settings?.fpts_decimal || 0;
      return {
        roster_id: r.roster_id,
        user_id: r.owner_id,
        name: user.metadata?.team_name || user.display_name || "Unknown",
        rec: `${wins}-${losses}`,
        wins,
        losses,
        pts: `${fpts}.${String(fptsD).padStart(2, "0")}`,
        pts_num: fpts + fptsD / 100,
        me: r.owner_id === myUserId,
      };
    })
    .sort((a, b) => b.wins - a.wins || b.pts_num - a.pts_num)

```

```

        .map((r, i) => ({ ...r, rank: i + 1 }));

    return standings;
}
}

/* =====
██████ YAHOO ██████
Yahoo uses OAuth 2.0 with PKCE.
Flow: Authorize → Callback → Token Exchange → Refresh
Docs: https://developer.yahoo.com/oauth2/guide/
===== */

class YahooClient {
  constructor(accessToken) {
    this.accessToken = accessToken;
    this.base = "https://fantasysports.yahooapis.com/fantasy/v2";
  }

  static generatePKCE() {
    const verifier = crypto.randomBytes(32).toString("base64url");
    const challenge = crypto.createHash("sha256").update(verifier).digest("base64");
    return { verifier, challenge };
  }

  static getAuthUrl(state, codeChallenge) {
    const params = new URLSearchParams({
      client_id: YAHOO_CLIENT_ID,
      redirect_uri: YAHOO_REDIRECT_URI,
      response_type: "code",
      scope: "fspt-r", // fantasy read
      code_challenge: codeChallenge,
      code_challenge_method: "S256",
      state,
    });
    return `https://api.login.yahoo.com/oauth2/request_auth?${params}`;
  }

  static async exchangeCode(code, verifier) {
    const res = await fetch("https://api.login.yahoo.com/oauth2/get_token", {
      method: "POST",
      headers: {
        "Content-Type": "application/x-www-form-urlencoded",
        "Authorization": "Basic " + Buffer.from(`${YAHOO_CLIENT_ID}:${YAHOO_CLIENT_SECRET}`).toString("base64"),
      },
      body: new URLSearchParams({
        grant_type: "authorization_code",
        code,
        redirect_uri: YAHOO_REDIRECT_URI,
        code_verifier: verifier,
      }),
    });
    const json = await res.json();
    return json;
  }
}

```

```

        redirect_uri: YAHOO_REDIRECT_URI,
        code_verifier: verifier,
    )),
    });
    if (!res.ok) {
        const err = await res.text();
        throw new Error(`Yahoo token exchange failed: ${err}`);
    }
    return res.json();
    // Returns: { access_token, refresh_token, expires_in, token_type }
}

static async refreshToken(refreshToken) {
    const res = await fetch("https://api.login.yahoo.com/oauth2/get_token", {
        method: "POST",
        headers: {
            "Content-Type": "application/x-www-form-urlencoded",
            "Authorization": "Basic " + Buffer.from(`${YAHOO_CLIENT_ID}:${YAHOO_CLIENT_SECRET}`).toString('base64'),
        },
        body: new URLSearchParams({
            grant_type: "refresh_token",
            refresh_token: refreshToken,
        }),
    });
    if (!res.ok) throw new Error("Yahoo token refresh failed");
    return res.json();
}

async get(path) {
    const res = await fetch(`${this.base}${path}?format=json`, {
        headers: { Authorization: `Bearer ${this.accessToken}` },
    });
    if (res.status === 401) throw new Error("yahoo_token_expired");
    if (!res.ok) throw new Error(`Yahoo API error: ${res.status}`);
    return res.json();
}

async getUserLeagues(season = new Date().getFullYear()) {
    const d = await this.get(`/users;use_login=1/games;game_codes=nfl;seasons=${season}`);
    // Navigate Yahoo's XML-turned-JSON structure
    const games = d?.fantasy_content?.users?.[0]?.user?.[1]?.games;
    if (!games) return [];
    return Object.values(games)
        .filter(g => g?.leagues)
        .flatMap(g => Object.values(g.leagues).filter(l => l?.league));
}

```

```

async getLeagueStandings(leagueKey) {
  const d = await this.get(`/league/${leagueKey}/standings`);
  const standings = d?.fantasy_content?.league?.[1]?.standings?.[0]?.teams;
  if (!standings) return [];
  return Object.values(standings)
    .filter(t => t?.team)
    .map((t, i) => {
      const info = t.team[0];
      const stats = t.team[2]?.team_standings;
      return {
        rank: parseInt(stats?.rank) || i + 1,
        name: info?.find?.(x => x?.name)?.name || "Unknown",
        rec: `${stats?.outcome_totals?.wins || 0}-${stats?.outcome_totals?.losses || 0}`,
        pts: stats?.points_for?.toFixed(2) || "0.00",
        pts_num: parseFloat(stats?.points_for) || 0,
        me: false, // set based on user's team_key after auth
      };
    });
}

```

```

/* =====
██████ ESPN ██████
ESPN does not have a public OAuth API for fantasy data.
The workaround is cookie-based: users provide their ESPN_S2
and SWID cookies (found in browser dev tools after login).
Your backend then uses these to authenticate API calls.
Docs: https://github.com/cwendt94/espn-api (community)
===== */

```

```

class ESPNClient {
  constructor(espnS2, swid) {
    this.cookies = `espn_s2=${espnS2}; SWID=${swid}`;
    this.base = "https://lm-api-reads.fantasy.espn.com/apis/v3/games/ffl";
  }

  async getLeague(leagueId, season = new Date().getFullYear()) {
    const url = `${this.base}/seasons/${season}/segments/0/leagues/${leagueId}`;
    const res = await fetch(`${url}?view=mStandings&view=mTeam`, {
      headers: { Cookie: this.cookies, "X-Fantasy-Source": "kona" },
    });
    if (!res.ok) throw new Error(`ESPN API error: ${res.status}`);
    return res.json();
  }

  async buildStandings(leagueId, myTeamId, season) {
    const data = await this.getLeague(leagueId, season);
    const teams = data?.teams || [];
  }
}

```

```

const members = data?.members || [];

const memberMap = Object.fromEntries(members.map(m => [m.id, m]));

return teams
  .map(t => ({
    rank:      t.rankCalculatedFinal || t.rankFinal || 99,
    name:      t.name || t.abbrev || "Team",
    rec:       `${t.record?.overall?.wins||0}-${t.record?.overall?.losses||0}`,
    pts:       (t.points || 0).toFixed(2),
    pts_num:   t.points || 0,
    me:        t.id === myTeamId,
  }))
  .sort((a, b) => a.rank - b.rank);
}
}

/* =====
EXPRESS ROUTES
===== */
const sleeper = new SleeperClient();

// — SLEEPER CONNECT (no OAuth, username only) —
router.post("/auth/sleeper/connect", async (req, res) => {
  const { username, userId: appUserId } = req.body;
  if (!username) return res.status(400).json({ error: "username required" });

  try {
    // 1. Resolve username to Sleeper user
    const sleeperUser = await sleeper.getUser(username);

    // 2. Get their leagues
    const leagues = await sleeper.getUserLeagues(sleeperUser.user_id);
    if (!leagues.length) return res.status(404).json({ error: "No NFL leagues fou

    // 3. Save platform connection to Supabase
    await supabase.from("platform_connections").upsert({
      user_id:      appUserId,
      platform:     "sleeper",
      platform_user_id: sleeperUser.user_id,
      platform_username: sleeperUser.username,
      updated_at:   new Date().toISOString(),
    });

    res.json({
      ok: true,
      sleeperUserId: sleeperUser.user_id,

```

```

        displayName: sleeperUser.display_name,
        leagues: leagues.map(l => ({
            id: l.league_id,
            name: l.name,
            size: l.total_rosters,
            scoring: l.scoring_settings?.rec > 0 ? (l.scoring_settings.rec === 1 ? "P
        })),
    });
} catch (e) {
    res.status(400).json({ error: e.message });
}
});

```

// — YAHOO AUTHORIZE —————

```

router.get("/auth/yahoo/authorize", async (req, res) => {
    const { userId } = req.query;
    const { verifier, challenge } = YahooClient.generatePKCE();
    const state = crypto.randomBytes(16).toString("hex");

    // Store PKCE verifier and state in Supabase temporarily (expires in 10 min)
    await supabase.from("oauth_state").upsert({
        state,
        platform: "yahoo",
        user_id: userId,
        verifier,
        expires_at: new Date(Date.now() + 10 * 60 * 1000).toISOString(),
    });

    res.redirect(YahooClient.getAuthUrl(state, challenge));
});

```

// — YAHOO CALLBACK —————

```

router.get("/auth/yahoo/callback", async (req, res) => {
    const { code, state } = req.query;
    if (!code || !state) return res.status(400).send("Missing code or state");

    try {
        // 1. Retrieve PKCE verifier
        const { data: oauthRow, error } = await supabase
            .from("oauth_state")
            .select("*")
            .eq("state", state)
            .eq("platform", "yahoo")
            .single();
        if (error || !oauthRow) return res.status(400).send("Invalid or expired OAuth
        if (new Date(oauthRow.expires_at) < new Date()) return res.status(400).send("

```

```

// 2. Exchange code for tokens
const tokens = await YahooClient.exchangeCode(code, oauthRow.verifier);

// 3. [3] VAULT ENCRYPT - store tokens as encrypted secrets, save only UUIDs
const [accessSecretId, refreshSecretId] = await Promise.all([
  vaultCreate(tokens.access_token, `yahoo_access_${oauthRow.user_id}_${Date.now()}`,
  vaultCreate(tokens.refresh_token, `yahoo_refresh_${oauthRow.user_id}_${Date.now()}`
]);

// Save only the Vault secret_id UUIDs - zero plaintext in DB
await supabase.from("platform_connections").upsert({
  user_id:      oauthRow.user_id,
  platform:     "yahoo",
  token_secret_id: accessSecretId, // Vault UUID - not the token
  refresh_secret_id: refreshSecretId, // Vault UUID - not the token
  token_expires_at: new Date(Date.now() + tokens.expires_in * 1000).toISOString(),
  updated_at:      new Date().toISOString(),
});

// 4. Clean up state row
await supabase.from("oauth_state").delete().eq("state", state);

// 5. Redirect back to app
res.redirect(`${APP_BASE_URL}/dashboard?connected=yahoo`);
} catch (e) {
  res.status(500).send(`OAuth error: ${e.message}`);
}
});

// — ESPN CONNECT (cookie-based) —————
router.post("/auth/espn/connect", async (req, res) => {
  const { espnS2, swid, leagueId, userId } = req.body;
  if (!espnS2 || !swid || !leagueId) return res.status(400).json({ error: "espnS2"

  try {
    const espn = new ESPNClient(espnS2, swid);
    const data = await espn.getLeague(leagueId);

    if (!data?.id) return res.status(401).json({ error: "Invalid ESPN credentials"

    // [3] VAULT ENCRYPT - ESPN_S2 is a session credential, treat like a password
    const espnSecretId = await vaultCreate(
      espnS2,
      `espn_s2_${userId}_${Date.now()}`,
      `ESPN_S2 cookie - user ${userId}`
    );
  }

```

```

// SWID is a public user identifier (non-sensitive) – stored plaintext is fine
await supabase.from("platform_connections").upsert({
  user_id:      userId,
  platform:     "espn",
  espn_secret_id: espnSecretId, // Vault UUID for ESPN_S2, not the cookie its
  espn_swid:     swid,           // non-sensitive, stored plain
  league_id:     String(leagueId),
  updated_at:    new Date().toISOString(),
});

res.json({ ok: true, leagueName: data.settings?.name || "ESPN League" });
} catch (e) {
  res.status(400).json({ error: e.message });
}
});

// — LIVE STANDINGS (all platforms) – Redis-cached + Vault-decrypted —
router.get("/league/standings", async (req, res) => {
  const { userId, leagueId } = req.query;

  try {
    // [2] Redis cache – check before hitting platform API
    const cacheKey = `ssff:standings:${leagueId}`;
    const cached = await redis.get(cacheKey).catch(() => null);
    if (cached) {
      const parsed = typeof cached === "string" ? JSON.parse(cached) : cached;
      return res.json({ standings: parsed, source: "cache", cachedAt: parsed._cac
    }

    // Get platform connection from Supabase
    const { data: conn } = await supabase
      .from("platform_connections")
      .select("platform, token_secret_id, refresh_secret_id, token_expires_at, es
      .eq("user_id", userId)
      .single();

    if (!conn) return res.status(404).json({ error: "No platform connected" });

    let standings = [];

    switch (conn.platform) {
      case "sleeper": {
        // Sleeper uses public API – no tokens to decrypt
        standings = await sleeper.buildStandings(leagueId, conn.platform_user_id)
        break;
      }
    }
  }
});

```

```

case "yahoo": {
  // [3] VAULT DECRYPT - read secret_id, get plaintext token only at query
  let accessToken = await vaultDecrypt(conn.token_secret_id);
  if (!accessToken) throw new Error("Yahoo token not found in Vault");

  // Token expired - refresh it and re-encrypt the new one
  if (new Date(conn.token_expires_at) < new Date()) {
    const refreshToken = await vaultDecrypt(conn.refresh_secret_id);
    if (!refreshToken) throw new Error("Yahoo refresh token not found in Va

    const refreshed = await YahooClient.refreshToken(refreshToken);
    accessToken = refreshed.access_token;

    // [3] Re-encrypt new access token in Vault, update expiry in DB
    await vaultUpdate(conn.token_secret_id, refreshed.access_token);
    await supabase.from("platform_connections").update({
      token_expires_at: new Date(Date.now() + refreshed.expires_in * 1000).
        updated_at:      new Date().toISOString(),
    }).eq("user_id", userId);
  }

  const yahoo = new YahooClient(accessToken);
  standings = await yahoo.getLeagueStandings(leagueId);
  break;
}

case "espn": {
  // [3] VAULT DECRYPT - ESPN_S2 cookie decrypted only at request time
  const espnS2 = await vaultDecrypt(conn.espn_secret_id);
  if (!espnS2) throw new Error("ESPN_S2 not found in Vault");

  const espn = new ESPNClient(espnS2, conn.espn_swid);
  standings = await espn.buildStandings(leagueId, conn.espn_team_id, new Da
  break;
}

default:
  return res.status(400).json({ error: `Unknown platform: ${conn.platform}`
}

// [2] REDIS CACHE WRITE - 5 min TTL
// 100 users in the same league now share this one live fetch
const toCache = { ...standings, _cachedAt: new Date().toISOString() };
await redis.set(cacheKey, JSON.stringify(toCache), { ex: 300 }).catch(() => n

res.json({ standings, source: conn.platform, liveAt: new Date().toISOString()
} catch (e) {

```

```

    res.status(500).json({ error: e.message });
  }
});

// — LIVE ROSTER —————
router.get("/league/roster", async (req, res) => {
  const { userId, leagueId } = req.query;

  try {
    // [2] Redis cache — roster changes infrequently, 2 min TTL is safe
    const cacheKey = `ssff:roster:${userId}:${leagueId}`;
    const cached = await redis.get(cacheKey).catch(() => null);
    if (cached) {
      const parsed = typeof cached === "string" ? JSON.parse(cached) : cached;
      return res.json({ roster: parsed, source: "cache" });
    }

    const { data: conn } = await supabase
      .from("platform_connections")
      .select("platform, platform_user_id, espn_secret_id, espn_swid, token_secret")
      .eq("user_id", userId).single();

    if (!conn) return res.status(404).json({ error: "No platform connected" });

    let roster = [];

    if (conn.platform === "sleeper") {
      // [2] Cache the Sleeper player DB separately — it's ~5MB and changes once/
      const playerCacheKey = "ssff:players:nfl";
      let players = await redis.get(playerCacheKey).catch(() => null);
      if (players) {
        players = typeof players === "string" ? JSON.parse(players) : players;
      } else {
        players = await sleeper.getPlayerInfo();
        await redis.set(playerCacheKey, JSON.stringify(players), { ex: 86400 }).catch();
      }

      const rosters = await sleeper.getLeagueRosters(leagueId);
      const myRoster = rosters.find(r => r.owner_id === conn.platform_user_id);
      if (!myRoster) return res.status(404).json({ error: "Roster not found" });

      roster = (myRoster.players || []).map(pid => {
        const p = players[pid] || {};
        return {
          id: pid,
          name: p.full_name || pid,
          position: p.position,
        };
      });
    }
  }
});

```

```

        team:      p.team,
        status:    p.injury_status || "active",
        starter:   myRoster.starters?.includes(pid),
    };
    });
}

if (conn.platform === "espn") {
    // [3] Vault-decrypt ESPN_S2 before use
    const espnS2 = await vaultDecrypt(conn.espn_secret_id);
    if (!espnS2) return res.status(401).json({ error: "ESPN credentials not fou
    // ESPN roster fetch would go here - same ESPNClient pattern as standings
}

if (conn.platform === "yahoo") {
    // [3] Vault-decrypt Yahoo access token before use
    let token = await vaultDecrypt(conn.token_secret_id);
    if (!token) return res.status(401).json({ error: "Yahoo token not found" })
    // Yahoo roster fetch would go here - YahooClient.get(/team/.../roster)
}

// [2] Cache result for 2 minutes
await redis.set(cacheKey, JSON.stringify(roster), { ex: 120 }).catch(() => nu

res.json({ roster, source: conn.platform });
} catch (e) {
    res.status(500).json({ error: e.message });
}
});

module.exports = router;

```

```
/*
```

```
SUPABASE SCHEMA - v3 (Vault columns, no plaintext tokens)
```

```
create extension if not exists supabase_vault;
```

```
create table if not exists platform_connections (
    user_id          uuid references users(id) on delete cascade,
    platform         text not null,          -- 'sleeper' | 'yahoo' | 'espn'
    platform_user_id text,                  -- Sleeper user_id
    platform_username text,                 -- Sleeper username

    -- [3] VAULT: token UUIDs only - plaintext never in DB
    token_secret_id  uuid,                  -- Yahoo access token (Vault)

```

```

refresh_secret_id uuid,          -- Yahoo refresh token (Vault)
token_expires_at  timestampz,    -- expiry (non-sensitive metadata)
espn_secret_id    uuid,          -- ESPN_S2 cookie (Vault)
espn_swid         text,          -- SWID is public, stored plain

league_id         text,
espn_team_id      int,
updated_at        timestampz,
primary key (user_id, platform)
);

create table if not exists oauth_state (
    state          text primary key,
    platform       text,
    user_id        uuid,
    verifier       text,
    expires_at     timestampz
);

-- Vault RPCs (run once):
create or replace function vault_decrypt_secret(secret_id uuid)
returns jsonb language sql security definer as $$
    select jsonb_build_object('decrypted_secret', vault.decrypted_secret(secret_i
$$;

create or replace function vault_create_secret(
    secret text, name text, description text default ''
) returns uuid language sql security definer as $$
    select vault.create_secret(secret, name, description);
$$;

create or replace function vault_update_secret(
    secret_id uuid, new_secret text
) returns void language sql security definer as $$
    update vault.secrets set secret = new_secret where id = secret_id;
$$;

```

ENVIRONMENT VARIABLES

```

SUPABASE_URL          - Supabase project URL
SUPABASE_SERVICE_KEY  - Service role key (not anon)
YAHOO_CLIENT_ID       - developer.yahoo.com app client ID
YAHOO_CLIENT_SECRET   - developer.yahoo.com app secret
YAHOO_REDIRECT_URI    - https://yourdomain.com/api/auth/yahoo/callback
APP_BASE_URL          - https://yourdomain.com
REDIS_URL             - Upstash Redis REST URL

```

REDIS_TOKEN - Upstash Redis REST token

MOUNT IN EXPRESS APP

```
const platform = require("./ssffmvp_platform_integration");
app.use("/api", platform);
```

PACKAGES

```
npm i express @supabase/supabase-js @upstash/redis
```

```
*/
    user_id      uuid,
    verifier      text,
    expires_at    timestampz
  );
```

MOUNT IN YOUR EXPRESS APP:

```
const platformRoutes = require("./platform-integration");
app.use("/api", platformRoutes);
```

ENVIRONMENT VARIABLES:

SUPABASE_URL	- your supabase project url
SUPABASE_SERVICE_KEY	- service role key (not anon)
YAHOO_CLIENT_ID	- from developer.yahoo.com
YAHOO_CLIENT_SECRET	- from developer.yahoo.com
YAHOO_REDIRECT_URI	- https://yourdomain.com/api/auth/yahoo/callback
APP_BASE_URL	- https://yourdomain.com

FRONTEND "CONNECT" BUTTON FLOW:

Sleeper: POST /api/auth/sleeper/connect { username, userId }
 → returns leagues array for user to pick from

Yahoo: GET /api/auth/yahoo/authorize?userId=X
 → browser redirect to Yahoo login
 → Yahoo redirects to /callback
 → app redirects to /dashboard?connected=yahoo

ESPN: POST /api/auth/espn/connect { espnS2, swid, leagueId, userId }
 ESPN_S2 and SWID found in browser cookies at espn.com/fantasy

*/

